

```
$ agno learn --chapter 3  
> loading modules ... database / history / memory  
> examples/03_storage_memory ... 10 files found [OK]
```

存储与记忆

DATABASE · HISTORY · MEMORY

让 Agent 记得你是谁 — 不止这一轮对话。■

CHAPTER 03 · SQLITE · SESSION · USER MEMORY

```
$ cat 01_three_concepts.md
```

三个词分不清，这章就白学

概念	存什么	回答的问题
Session History	对话消息记录	用户刚才具体说了什么？（连续上下文用）
Session State	跨 run 的内部状态（购物车、计数器）	流程进行到哪一步了？
User Memory	从对话提取的用户事实与偏好	系统从交互中学到了哪些可复用事实？

history 管「刚才」，state 管「进度」，memory 管「长期」。

```
$ cat 02_session_storage.md
```

挂上 db, 一切自动落库

```
db = SqliteDb(db_file="my_agent.db")
agent = Agent(model=..., db=db)
```

> 完整 run、消息、状态、元数据自动存进 **agno_sessions** 表

表不存在会自动创建, 不用手写 schema

> 支持 13+ 数据库: 开发用 **SQLite**, 生产用 **PostgreSQL**

[!] 最常见的误解

配了 db 只代表「保存」—— **不代表**旧消息会自动发给模型。要让模型看到历史, 看下一页的条件链。

```
$ cat 03_history_chain.py · examples/03_storage_memory/02_multi_user.py
```

模型看到历史，需要四个条件同时成立

```
customer_service_agent = Agent(
    model=DeepSeek(...),
    db=db,
    update_memory_on_run=True,
    add_history_to_context=True, # ③ 历史放回上下文
    num_history_runs=3,         # ④ 带最近 3 轮
)

# ① 每个用户固定 session_id ② 固定 user_id
```

- > ① 稳定的 **session_id**
同一个会话的钥匙

- > ② 稳定的 **user_id**
同一个人的钥匙

- > ③ **add_history_to_context=True**
默认不放回!

- > ④ **num_history_runs** 控制带几轮
默认 3; 越多越贵越慢, 官方建议从小起步

```
$ cat 04_auto_memory.md
```

自动记忆: `update_memory_on_run=True`

- > 每轮对话后，自动把**用户事实与偏好**提取进 `agno_memories` 表
"Sarah prefers email" 这类
- > 官方推荐用于**大多数场景**：可靠、可预期
- > 提取由模型参与 → **非确定**：不保证每条输入都生成记忆
- > 成本提醒：自动模式每轮多一次 `MemoryManager` 调用

记忆存进去后，后续对话会自动注入上下文 — Agent 就「记得」你了。

```
$ cat 05_agentic_memory.md
```

Agentic 记忆: 让 Agent 自己决定记不记

```
## enable_agentic_memory=True
```

主 Agent 获得 `update_user_memory` 工具, 自行决定何时创建、更新、删除记忆。

不再每轮无条件提取, 更依赖模型判断。

[!] 互斥警告 (官方强调)

和 `update_memory_on_run` 不要同时开: 同开不报错, 但 `agentic` 永远优先, 自动更新被静默忽略。

另外 `enable_user_memories` 是即将弃用的别名, 新代码别用。

二选一: 自动派 稳, `agentic` 派 活。混用两头不占。

```
$ sqlite3 my_agent.db "select * from agno_memories limit 1"
```

一条 memory 的解剖图

```
┌ MEMORY RECORD ───────────┐
│
│ memory_id : 9f3c...        # 主键
│ memory    : "用户偏好用邮件沟通" # 事实正文
│ topics    : ["沟通偏好"]    # 可能为空!
│ input     : "以后有事发我邮件..." # 触发它的原始输入
│ user_id   : "customer_001"   # 属于谁 ← 隔离单位
│ agent_id  : "support_bot"    # 谁记的
│ created_at : ...
│
```

topics 由模型生成, 可能为空 — 生产代码别无脑 `join(memory.topics)`。

```
$ cat 06_crud.py · examples/03_storage_memory/06_curd_mem.py
```

读写删的正确姿势

```
# 读: 某个用户的全部记忆
memories = agent.get_user_memories(user_id="user_food_lover")

# 删单条: 走 db 接口
agent.db.delete_user_memory(
    memory_id=memory_to_delete.memory_id,
    user_id="user_food_lover",
)
```

> ✗ `agent.delete_user_memory(...)`

这个方法不存在 — 你 03_manual.py 注释里就踩过

> ✓ `agent.db.delete_user_memory(...)`

2.7.3 的正确入口

> `memory_manager.clear()` 是清整张表

不是只清当前用户, 慎用

```
$ cat 07_isolation_bugs.md · 你的真实踩坑
```

user_id 是隔离的生命线

[!] 打错一个字母 = 换了个人

08_p1.py 先用 `customer_limig`, 后用 `customer_liming` — 系统当成两个用户, 第一次记的姓名/预算全部「消失」。

[!] 忘了传 `user_id`

会落到默认用户上 — 多个真实用户共享同一批记忆, 这就是「用户看到彼此内容」的首要排查项。

[!] 切用户没切 `session`

01_auto_memory.py 从 `user_001` 换到 `user_zhangsan` 时没传新 `session_id`, 自动 `session` 会「粘」在 Agent 实例上, 第二个用户的会话持久化不可靠。

共同教训

`user_id` / `session_id` 当作主键对待: 从可信来源 (登录态) 来, 不要手敲、不要复用。

```
$ cat 08_multi_agent.py · examples/03_storage_memory/07_safe_multi_person.py
```

多 Agent 共享一张表 ≠ 自动隔离

```
class AgentScopedMemoryManager(MemoryManager):
    """只读取当前 Agent 创建的用户记忆。"""

    def read_from_db(self, user_id=None):
        if self.db is None:
            return None
        memories = self.db.get_user_memories(
            user_id=user_id,
            agent_id=self.agent_id,    # ← 强制按 agent 过滤
        )
```

- > 默认 MemoryManager 读记忆只按 **user_id** 过滤
记录里有 agent_id ≠ 默认会按它过滤

- > 两个 Agent 同库同表同 user_id → 默认**共享**记忆

- > 覆写 read_from_db() 强制带上 agent_id
tests/test_safe_multi_tenant_memory.py 用真实 SQLite 验证过

\$ cat 09_cleanup.md

数据清理的粒度陷阱

操作	影响范围	什么时候用
db.delete_user_memory(...)	单条 (需 memory_id + user_id)	用户要求删除某条事实
memory_manager.clear()	整张表	几乎只用于本地重置，生产慎用
换 db_file	整个库	独立实验之间隔离 — 你 10 个示例共用一个 my_agent.db 就互相污染过

生产环境必须回答：谁的数据、留多久、怎么删。

```
$ cat 10_boundaries.md
```

记忆的边界，越早知道越好

> memory 不是权威数据源

它是模型生成的摘要，不能替代订单、病历、成绩 — 那些应该走业务系统

> 提取是非确定的

创建/合并/topics 都有模型参与，别写依赖「一定会记住」的逻辑

> 记忆会涨，token 也会涨

注入上下文的用户记忆持续增长，要定期审视清理

> history 越多越慢越贵

num_history_runs 从 3 起步，不够再加

```
$ ./quiz.sh --chapter 3
```

检查点四连问

[01] session history、session state、memory 三者各存什么？

提示：消息 / 内部状态 / 用户事实

[02] 配了 db，模型就能看到历史对话吗？需要哪几个条件？

提示：不能；session_id + user_id + add_history_to_context + num_history_runs

[03] 哪两个参数互斥？同时开会发生什么？

提示：agentic 永远优先，自动被忽略

[04] 删除某用户的一条记忆，正确的 API 是什么？

提示：不在 Agent 上，在 db 上

```
$ tail -n1 ch3_notes.md
```

db 管「存」, history 管「刚才」,
memory 管「长期」——
user_id 把它们隔开。■

下一章 · ch5 Teams: 一个 Agent 搞不定? 组个队。

按 [←](#) 返回复习 · [Home](#) 回到封面